BACKEND · FRONTEND · ANALYSIS & DESIGN

BUILDING SERVICE FLOW

화면에서 서버까지,
서비스의 흐름을 설계합니다.

문제를 피하기보다 원인을 끝까지 파고들고,
사용자 경험과 시스템 안정성을 함께 고민하는
풀스택 개발자 한재훈입니다.

about — zsh

```
$ whoami
  한재훈 / Han Jaehun
$ cat role.txt
  신입 개발자 · 풀스택
$ echo $CONTACT
  sksqlrlek@gmail.com
  github.com/sksqlrlek
```

CONTENTS

목차

분석·설계부터 개발·테스트·배포까지, 프로젝트 전 과정을 담았습니다.

01 **About Me**
개발자로서의 강점과 핵심 가치

02 **Tech Stack & Process**
기술 스택, 협업 방식, 개발 흐름

03 **Project 01 — Cinema Talk**
MVC 구조와 기본기 중심 프로젝트

04 **Project 02 — MyCrewSoft**
통합 그룹웨어 설계·구현 프로젝트

05 **Closing**

ABOUT ME · 01

개발자로서의 강점



핵심 키워드

문제 해결 · 서비스 흐름 · 꾸준한 성장

문제의 원인을 끝까지 파고들고, 분석·설계부터 직접 하는 개발자

단순히 코드를 작성하는 데 그치지 않고, 요구사항 분석과 설계부터 구현·테스트까지 전 과정을 직접 경험하며 서비스의 흐름을 이해하는 풀스택 개발자입니다.

01 원인을 추적하는 문제 해결력

에러를 회피하지 않고 로그·쿼리·흐름을 직접 따라가며 근본 원인을 찾고 재발을 막습니다.

02 분석·설계부터 운영까지 전 과정

요구사항 정의·ERD·화면설계서 등 산출물을 직접 작성하며 SDLC 전반을 경험했습니다.

03 백엔드·프론트엔드를 아우르는 구현

Java·Spring부터 JSP·Vue까지, 서버와 화면을 함께 다루며 서비스 전체를 완성합니다.

TECH STACK · 02

기술 스택 한눈에 보기

분석·설계 · 백엔드 · 프론트엔드 · 데이터 · 인프라 전 영역을 다루는 풀스택 역량.

Backend

- Java
- REST API
- Spring Boot
- Spring MVC
- MyBatis
- JPA

Frontend

- JavaScript
- jQuery
- React
- HTML/CSS
- Bootstrap

Database

- Oracle
- Redis
- ERD 설계

DevOps · Infra

- Docker
- Git

분석 · 설계

- 요구사항 분석
- 유스케이스
- ERD
- 화면설계서
- API 명세
- UML

협업 · 도구

- Git/GitHub
- Notion
- Figma
- Slack

DEVELOPMENT PROCESS · 04

분석부터 운영까지, 직접 수행했습니다

01

요구사항 분석

기능 정의 · 유스케이스 · 요구사항 정의서 작성.

OUTPUTS

- 요구사항 정의서
- 유스케이스 다이어그램
- 기능 목록표
- 단위 업무 정의서

02

설계

시스템 아키텍처 · ERD · 화면설계서 · API 명세.

OUTPUTS

- ERD
- 시스템 아키텍처 문서
- 화면 설계서 (Figma)
- API 명세서

03

개발

백엔드 · 프론트엔드 구현 · 코드 리뷰 · 형상관리.

OUTPUTS

- 백엔드 · 프론트엔드 코드
- Git 브랜치 전략 적용
- PR · 코드 리뷰
- 커밋 컨벤션 준수

04

테스트

단위/통합 테스트 · 정적분석 · 버그 수정.

OUTPUTS

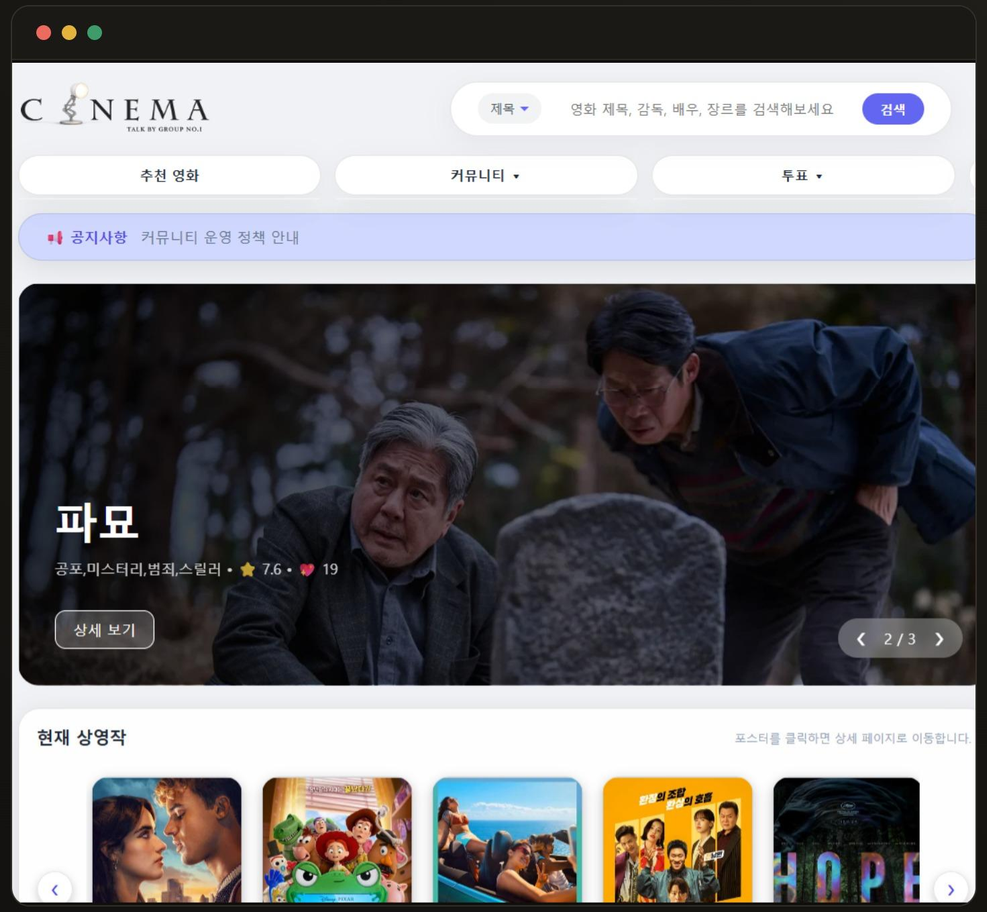
- 단위 테스트 코드
- 통합 테스트 코드
- 버그 리포트 · 수정 내역
- 테스트 결과 보고서

모든 프로젝트에서 **기획 단계부터 테스트까지 전 과정에 직접 참여**하며, 단순 구현을 넘어 소프트웨어 개발 생명주기(SDLC) 전반을 경험했습니다.

PROJECT 01 2026.02 – 2026.03 · 풀스택 / TA

01

Cinema Talk(영화 커뮤니티)



https://github.com/TaeSeungJeon/Cinema_Talk.git

개발 주요 사항

- `FrontController` + `Action` 인터페이스로 **MVC 패턴 직접 구현**
  (프레임워크 없이 `.do` URL 라우팅, Properties 기반 Command 매핑)
- **TMDB API 멀티스레드 배치 처리**로 영화/배우 데이터 자동 수집
  (`ExecutorService` 병렬 처리, 청크 단위 트랜잭션, 재시도 로직)
- **싱글톤 패턴**으로 `AppConfig` · `DBService` · `DAO` 계층 일관 관리
  (`SqlSessionFactory` 단일 인스턴스, XML 설정 로더 스레드 세이프 구현)
- **회원 기반 선호 장르 추천 시스템** 구현 (비로그인 랜덤 → 로그인 시 선호 장르 → 장르별 섹션 그룹핑)

기술 스택

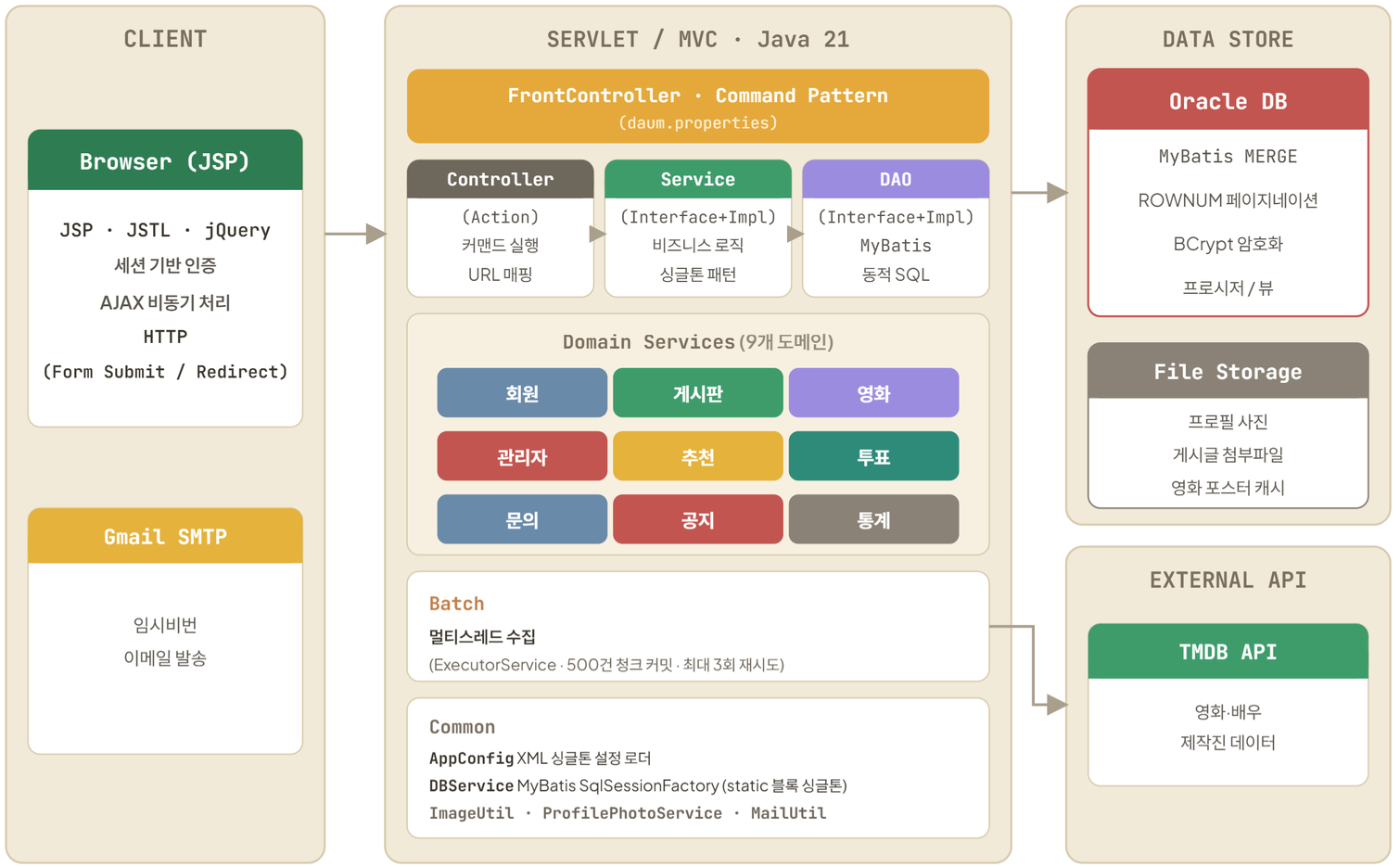
`Java 21` `Servlet/JSP` `MyBatis 3.5` `Oracle DB` `Maven` `Chart.js` `Tomcat 10`

한 줄 요약

순수 Servlet 기반 FrontController 패턴으로 MVC를 직접 구현한 영화 정보 · 커뮤니티 웹 서비스 — TMDB API 배치 수집, 장르 기반 추천, 관리자 통계 대시보드까지 풀스택 구현

PROJECT 01 · 시스템 설계

시스템 아키텍처



아키텍처 설명

구성 요소

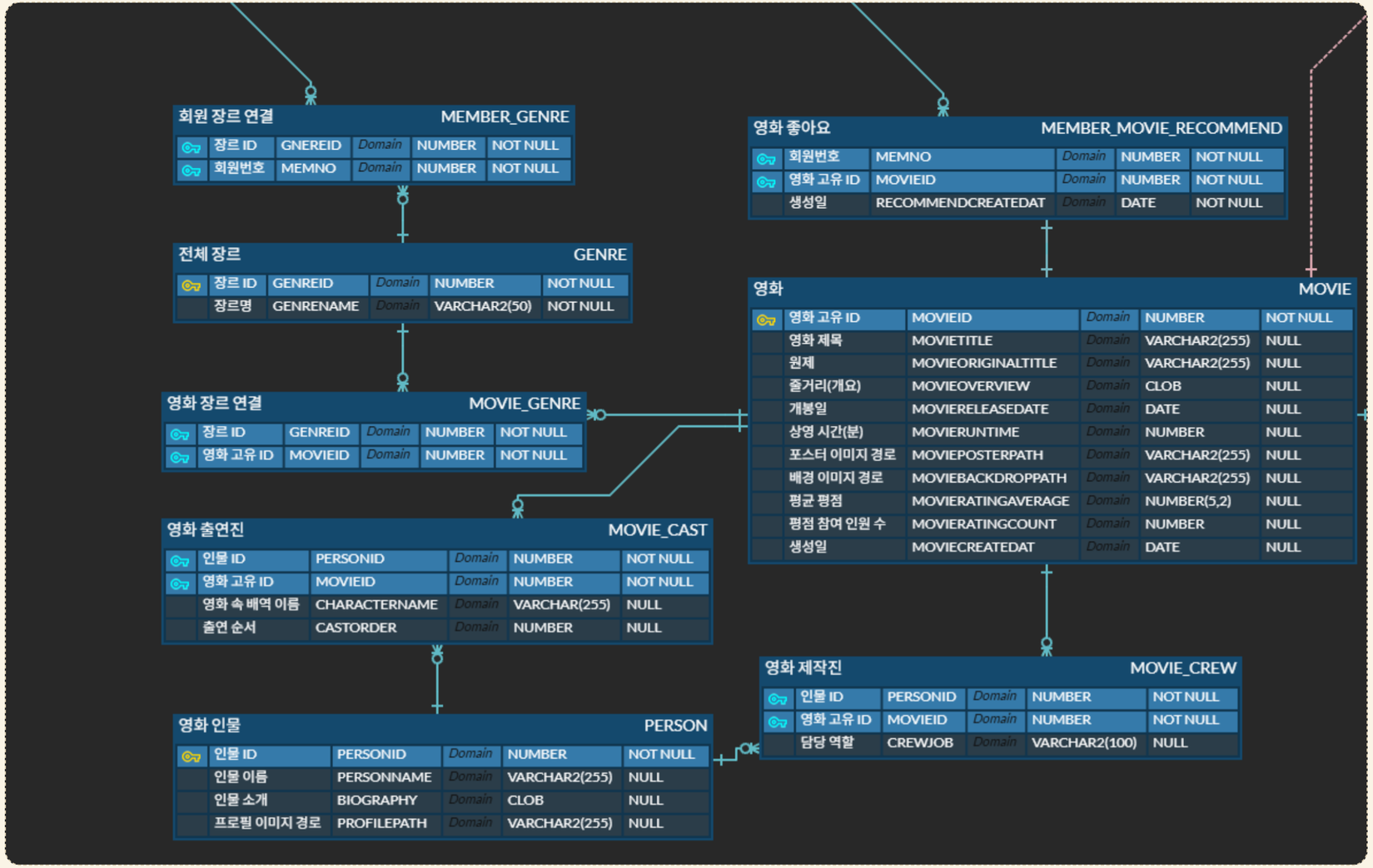
JSP · Servlet · MyBatis 기반 3계층 구조. FrontController가 모든 요청을 받아 Properties 파일로 커맨드를 매핑하고 Action 인터페이스 구현체를 Reflection으로 실행합니다. 외부 API(TMDB)는 서버 배치에서 직접 호출해 Oracle DB에 적재합니다.

설계 의도

Spring MVC 없이 MVC 패턴을 직접 구현해 프레임워크 동작 원리를 체득하는 데 목적을 뒀습니다.
AppConfig·DBService·DAO 계층에 싱글톤 패턴을 적용해 SqlSessionFactory 등 공유 리소스를 단일 인스턴스로 관리했습니다.

PROJECT 01 · 데이터 설계

ERD & DB 설계



주요 테이블

MOVIE	영화 관리
MOVIE_CREW	영화 제작진 관리
MOVIE_CAST	영화 출연진 관리
PERSON	인물 관리
MOVIE_GENRE	영화 장르 관리
GENRE	전체 장르 관리
MEMBER_GENRE	회원 선호 장르 관리
MEMBER_MOVIE_RECOMMEND	좋아요 관리

설계 포인트 —

3NF 정규화 · 역할 기반 분리 · 복합 PK

PROJECT 01 · 상세 설계

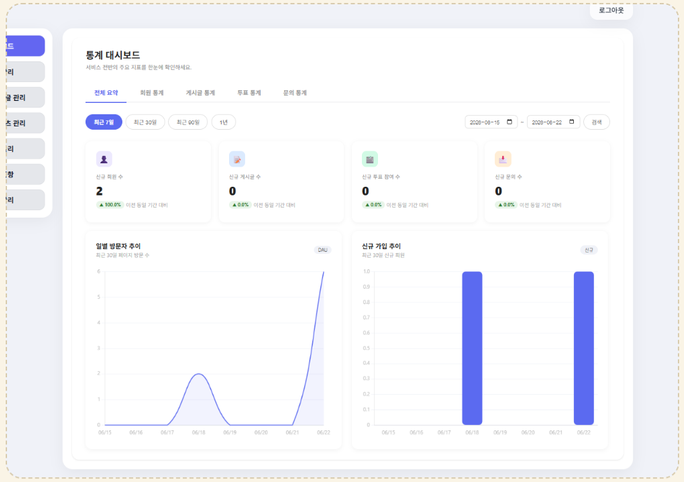
URL 명세 & 화면설계서

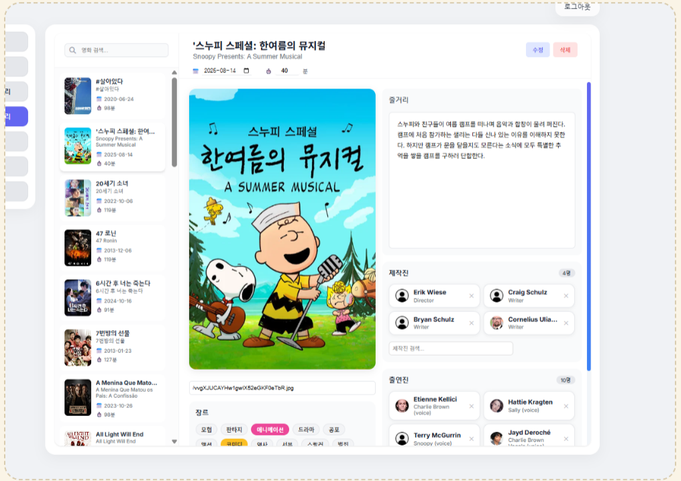
URL 명세 (.do 방식)

Method	Endpoint	설명
GET	/movieRec.do	영화 추천 목록 조회
GET	/movieDetail.do?movieId={id}	영화 상세 조회
POST	/loginOk.do	로그인 처리
POST	/registerOk.do	회원가입
POST	/boardOk.do	게시글 등록
GET	/boardList.do	게시글 목록 조회
POST	/pwdFindOk.do	임시 비밀번호 발급

화면설계서 (와이어프레임)



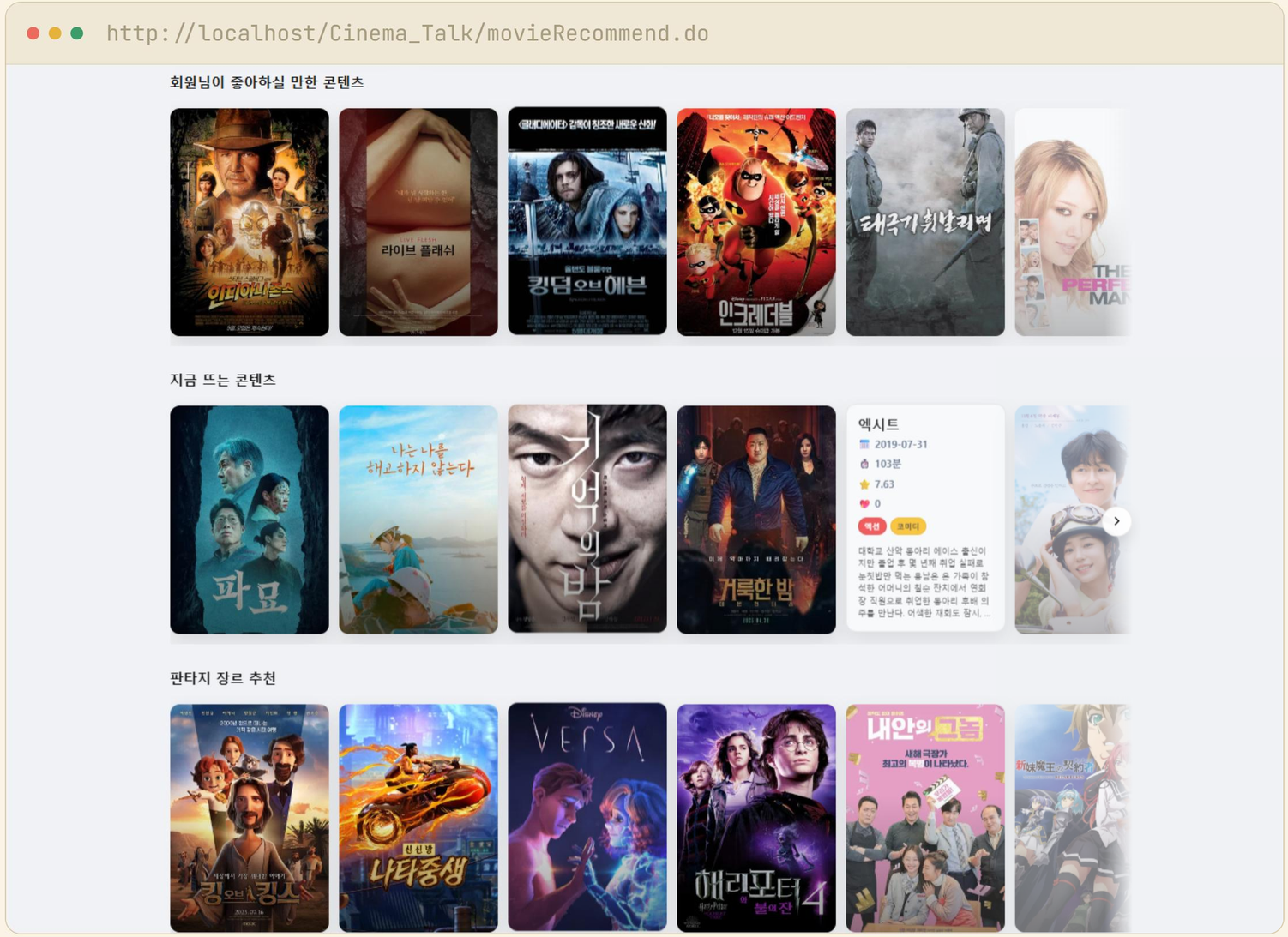






PROJECT 01 · 주요 기능

핵심 기능 상세



01 선호 장르 기반 영화 추천

로그인 여부 → 선호 장르 설정 여부에 따라 3단계로 분기합니다. 비로그인이면 랜덤, 선호 장르 미설정이면 랜덤, 선호 장르가 있으면 해당 장르 영화를 `DBMS_RANDOM`으로 셔플해 매번 다른 추천 결과를 제공합니다.

02 인기 영화 추천

평점 20% + 회원 추천수 80% 가중치로 점수를 계산해 상위 60개를 추린 뒤 `DBMS_RANDOM`으로 셔플합니다. 단순 정렬이 아닌 정규화 스코어링으로 인기 편향 없이 다양한 영화를 노출합니다

03 장르별 섹션 그룹핑

DB에서 장르별로 인기 상위 100개 → 랜덤 20개를 추린 뒤, 동일 영화 중복 노출을 `ROW_NUMBER() OVER (PARTITION BY movieId)`로 제거합니다. Java에서는 `LinkedHashMap`의 `computeIfAbsent`로 장르 순서를 유지하며 섹션 단위로 그룹핑합니다.

PROJECT 01 · 핵심 코드

코드 하이라이트

FrontController.java

```java
public class FrontController extends HttpServlet {
  @Override
  protected void service(HttpServletRequest request,
                         HttpServletResponse response) {
    String command = RequestURI.substring(contextPath.length());
    String value = prop.getProperty(command);
    //리플렉션으로 Action 생성 후 실행
    Class<?> url = Class.forName(url_2);
    action = (Action) url.getDeclaredConstructor().newInstance();
    forward = action.execute(request, response);
    if (forward.isRedirect()) {
        response.sendRedirect(forward.getPath());
    } else {
        RequestDispatcher dis =
            request.getRequestDispatcher(forward.getPath());
        dis.forward(request, response);
    }
  }
}
```

무엇을 하는 코드인가

모든 `.do` 요청을 하나의 서블릿이 받아, Properties 파일에서 URL에 해당하는 클래스명을 찾고 Reflection으로 인스턴스를 생성해 실행합니다. 결과에 따라 redirect 또는 forward로 응답합니다.

핵심 포인트

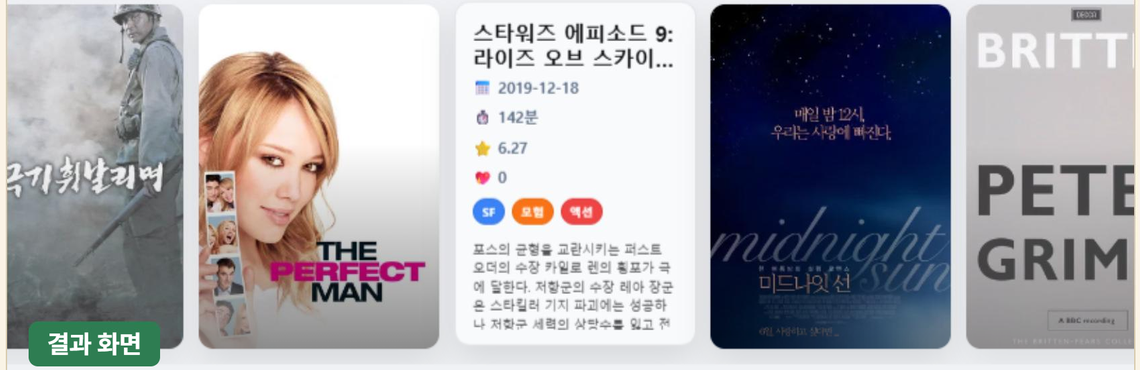
`Class.forName()+` `getDeclaredConstructor().newInstance()`로 컨트롤러를 동적 생성해, 새 기능 추가 시 Java 코드 수정 없이 **properties 파일 한 줄만 추가**하면 됩니다. Spring의 DispatcherServlet이 내부적으로 하는 일을 직접 구현한 구조입니다.

POINT — 프레임워크를 단순히 사용하는 데 그치지 않고, 내부 동작 원리를 직접 구현하며 이해했습니다.

PROJECT 01 · 화면 코드 리뷰

화면 ↔ 코드 매핑

http://localhost/Cinema_Talk/movieRecommend.do



결과 화면

list.jsp

```jsp
<%-- 카드 0.5초 호버 시 3D flip --%>
.card-inner {
  transition: transform .65s cubic-bezier(.2,.8,.2,1);
  transform-style: preserve-3d;
}
.movie-card.is-flipped .card-inner { transform: rotateY(180deg); }
.card-face.back { transform: rotateY(180deg); }

<%-- 장르별 동적 pill 색상 매핑 --%>
<c:forEach var="genre" items="${fn:split(movie.genreName, ',')}">
  <c:set var="g" value="${fn:trim(genre)}" />
  <span class="pill genre-${fn:replace(g,' ','-')}">${g}</span>
</c:forEach>
```

처리 흐름

Client → Controller → Service → Mapper → DB

상세 설명

추천 화면 요청 시 Controller가 `likeRecList`, `popularRecList`, `genreSections` 3개를 request scope에 담아 JSP로 전달합니다.

`movieCard.jsp`를 `jsp:include`로 분리해 카드 컴포넌트를 재사용했습니다.

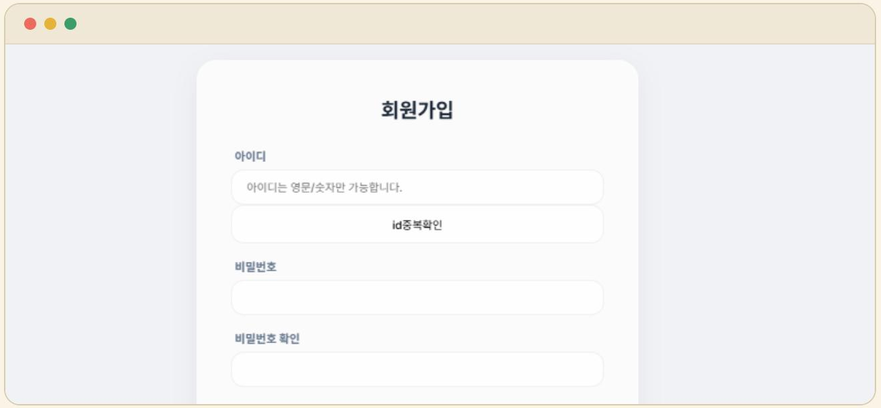
카드 앞면은 TMDB 포스터 이미지, 뒷면은 제목·개봉일·상영시간·평점·추천수·장르 pill을 표시합니다. 마우스 오버 0.5초 후 CSS `rotateY(180deg)`로 카드가 뒤집히며, 장르별 고유 색상 pill은 `genre-${장르명}` 클래스로 동적 매핑했습니다.

캐러셀은 드래그 스크롤과 좌우 버튼을 지원하며, 중앙에서 멀어질수록 `opacity`와 `blur`를 점진적으로 적용해 포커스 카드를 시각적으로 강조합니다.

PROJECT 01 · 화면 흐름

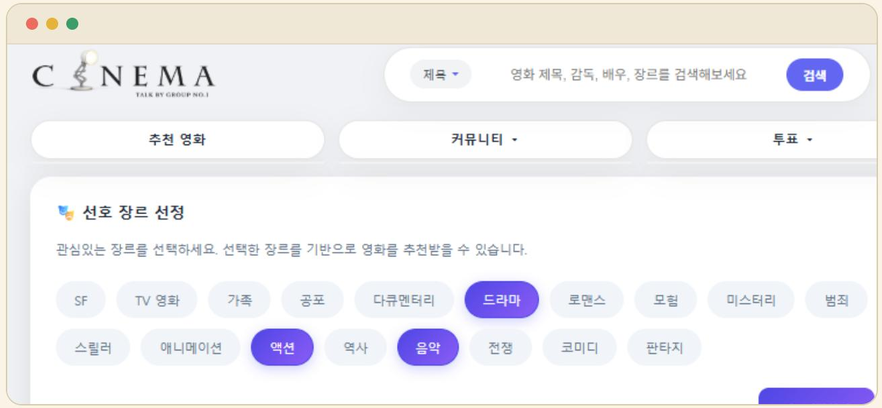
기능 시나리오

회원가입 후 선호 장르를 설정하고 영화를 탐색하는 핵심 사용자 흐름입니다.



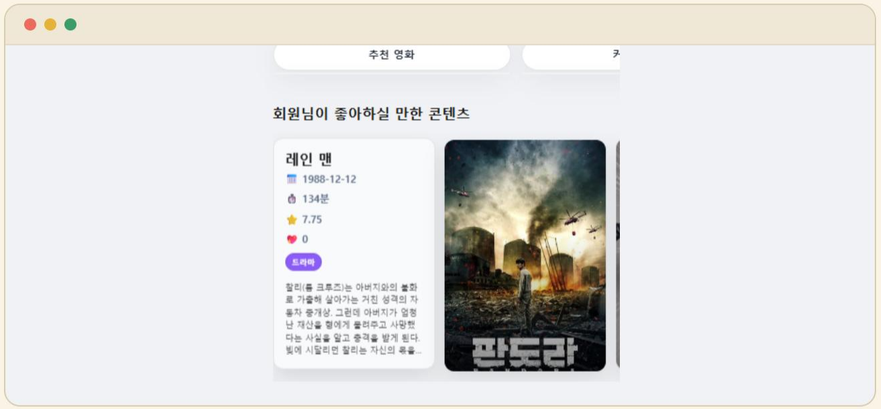
STEP 1 회원가입

비밀번호·이름·전화번호 입력 후 가입합니다. 아이디 중복확인과 BCrypt 암호화가 서버에서 처리됩니다.



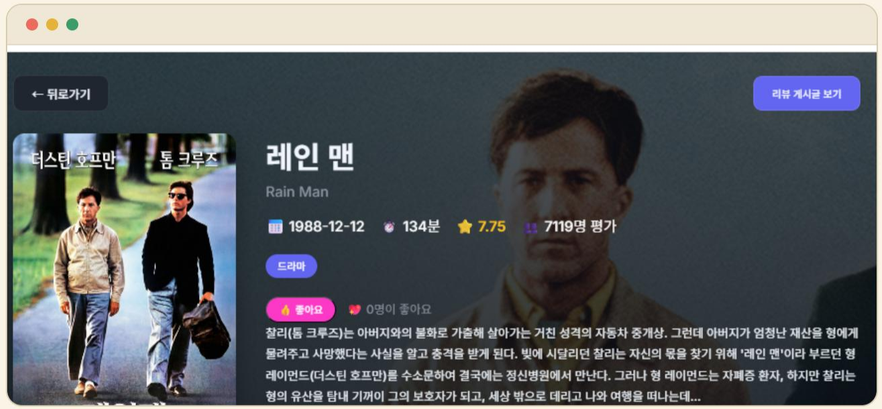
STEP 2 선호 장르 설정

가입 직후 장르 선택 화면으로 이동합니다. 선택한 장르는 `MEMBER_GENRE` 테이블에 저장되어 추천 알고리즘의 기반 데이터가 됩니다.



STEP 3 영화 탐색

추천 목록에서 영화를 클릭하면 포스터·평점·줄거리 정보를 확인합니다. 카드 뒤집기로 미리보기 후 상세로 진입합니다.



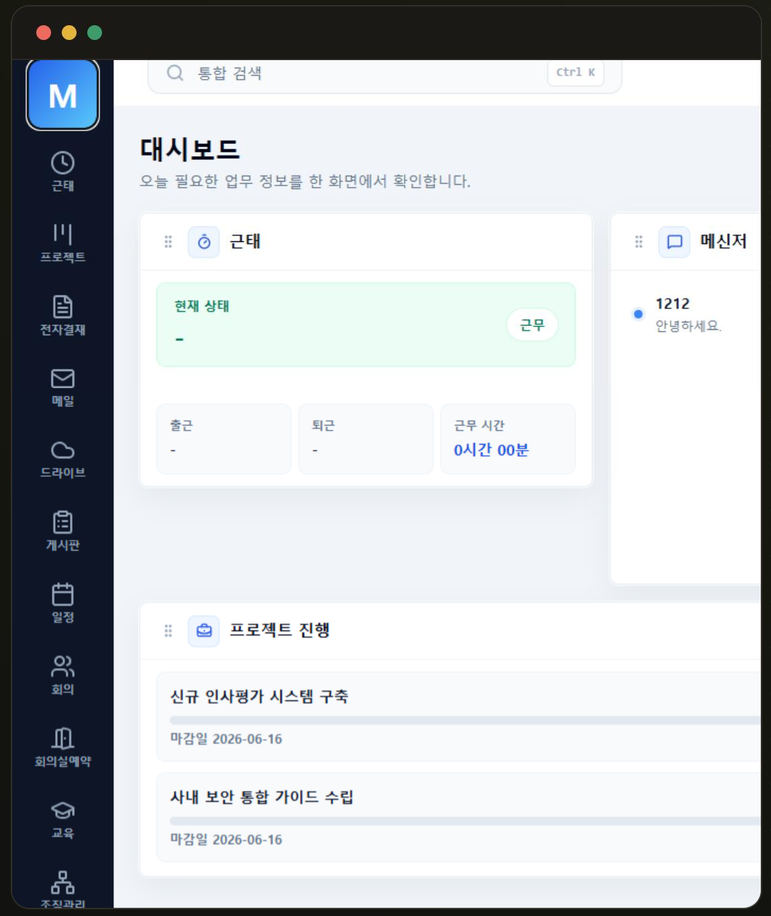
STEP 4 영화 상세 페이지

출연진·감독 정보와 줄거리 전문을 확인합니다. 좋아요 기능으로 `MEMBER_MOVIE_RECOMMEND` 테이블에 이력이 저장됩니다.

PROJECT 02 2026.05 – 2026.06 · 풀스택 / AA

02

MyCrewSoft (그룹웨어)



https://github.com/myCrewSoft/mycrew-backend.git

개발 주요 사항

— AA로서 **백엔드 공통 아키텍처 설계** 및 협업 가이드 작성·배포로 **4인 팀 개발 표준 정립**

— `LiveKit` 녹취 → `Whisper STT` → `LLM` **회의록 자동 생성** →
전자결재 기안까지 자동화 구현 (팩토리 메서드 + 템플릿 메서드 패턴)

— `@TransactionalEventListener` + `@Async` 이벤트 아키텍처로 **SSE 실시간 알림 시스템** 구현

— 대시보드 12종 위젯, 6개 도메인 통합검색, WebSocket 메신저, 공휴일 API 연동 등 다수 도메인 추가 구현

기술 스택

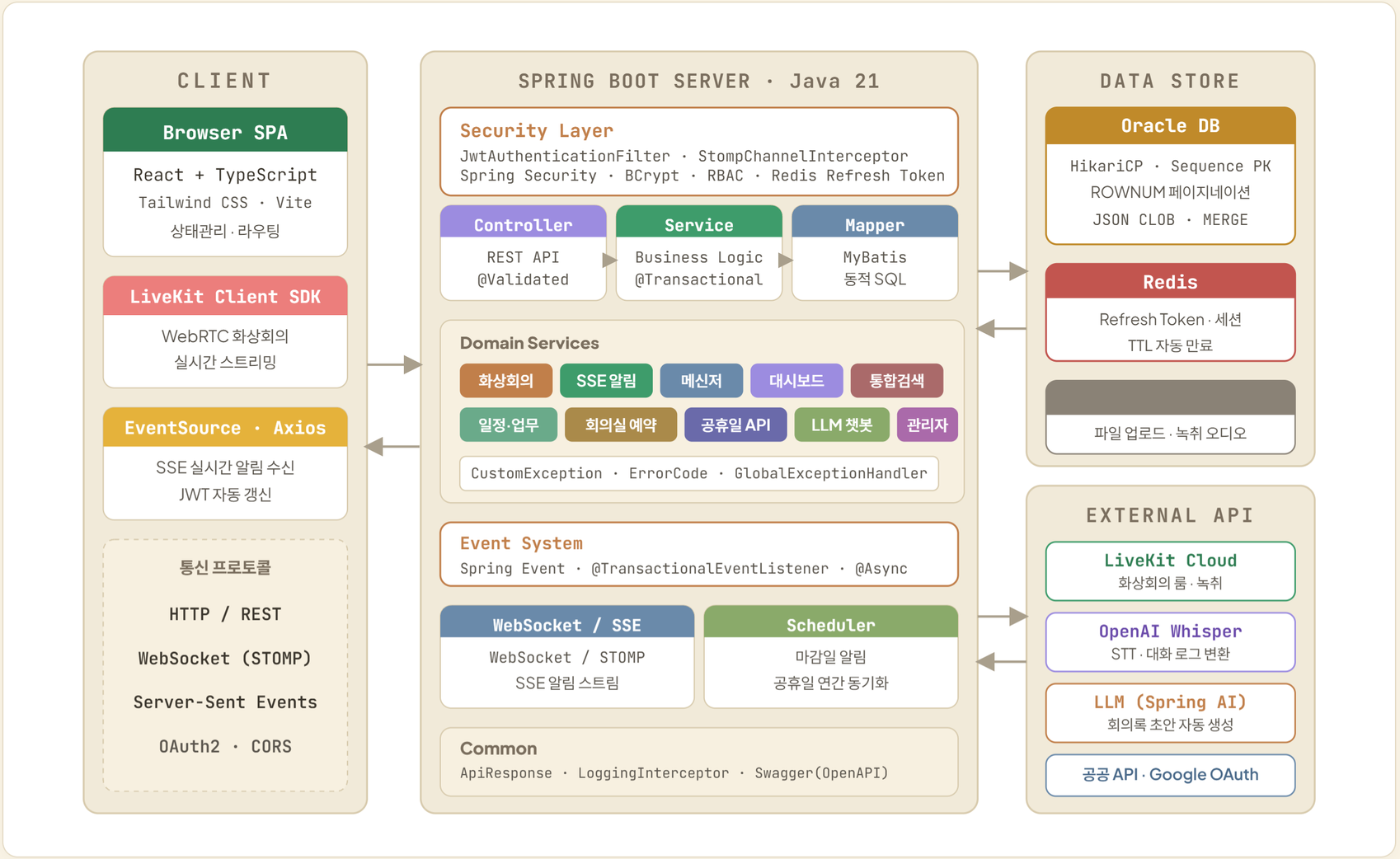
`Java 21` `Spring Boot 3.5.14` `Spring Security` `JWT` `WebSocket` `SEE` `LiveKit`
`React` `TypeScript` `Tailwind CSS` `Vite` `FullCalendar`

한 줄 요약

Spring Boot · React 기반 통합 그룹웨어 — AA 역할로 공통 아키텍처 설계부터 화상회의 AI 회의록 자동생성, 실시간 알림·메신저 등 10개 이상 도메인 풀스택 구현

PROJECT 02 · 시스템 설계

시스템 아키텍처



아키텍처 설명

구성 요소

React SPA → Spring Boot → Oracle DB / Redis 3계층 구조. 실시간 통신은 WebSocket(STOMP)·SSE, 화상회의는 LiveKit, AI 처리는 Whisper·LLM으로 분리했습니다.

설계 의도

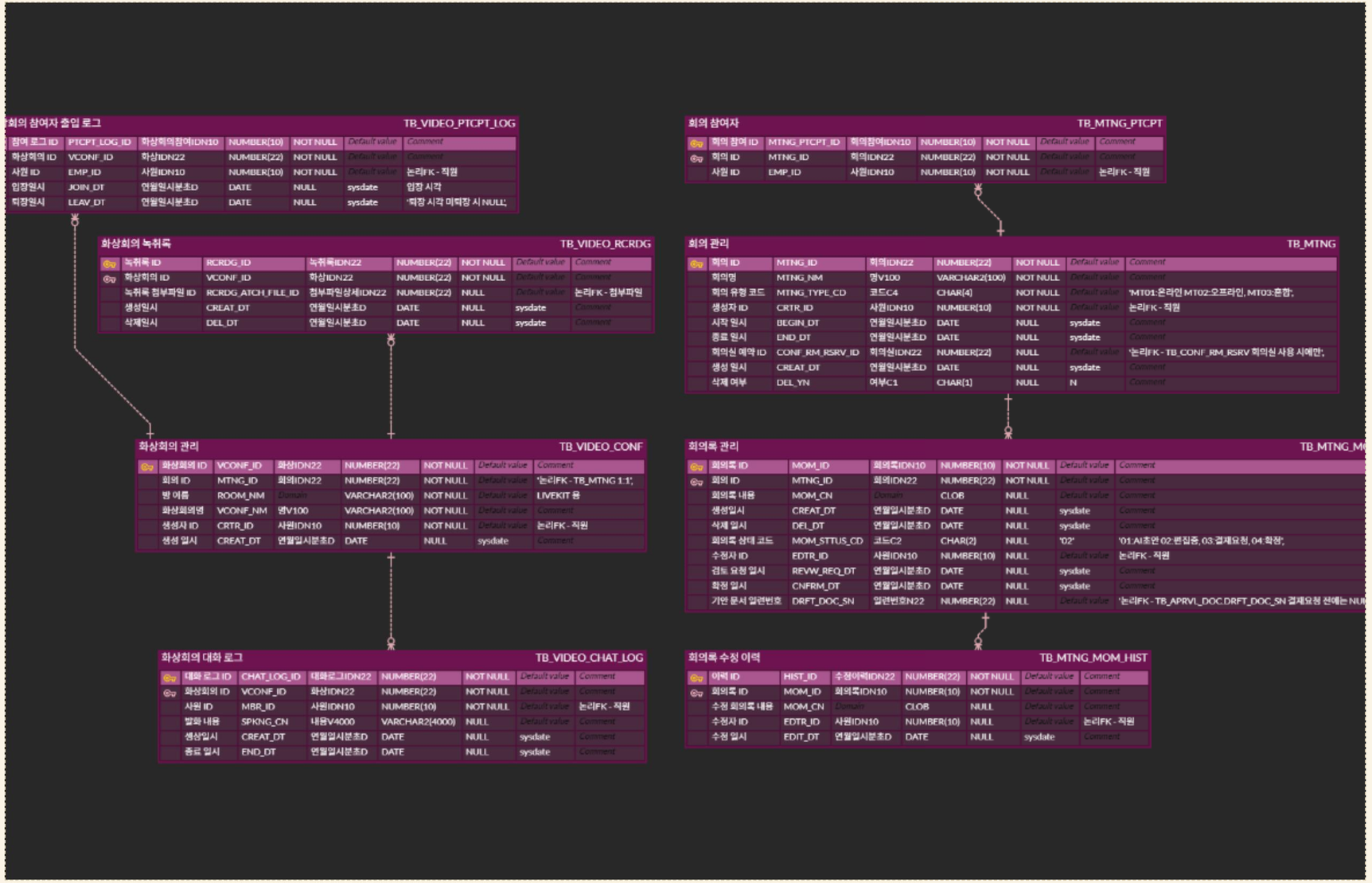
도메인 간 강결합을 막기 위해 Spring Event 기반 비동기 구조를 채택했습니다. HTTP와 WebSocket 각각에 JWT 인증을 적용해 보안 일관성을 확보했습니다.

팀 협업 방식

- GitHub(feature 브랜치 전략)
- Readmine(WBS·회의록)
- AA 협업 가이드

PROJECT 02 · 데이터 설계

ERD & DB 설계



주요 테이블

TB_MTNG	통합 회의 관리(오프라인/온라인)
TB_MTNG_PTCPT	회의 참여자 관리
TB_MTNG_MOM	회의록 관리(결재 테이블 논리FK)
TB_MTNG_MOM_HIST	회의록 수정 이력
TB_VIDEO_CONF	화상회의 관리
TB_VIDEO_PTCPT_LOG	화상회의 참여 이력
TB_VIDEO_RCRDG	화상회의 녹취록
TB_VIDEO_CHAT_LOG	화상회의 대화 로그

설계 포인트 —

3NF 정규화 · 1:N 관계 · ID 인덱스

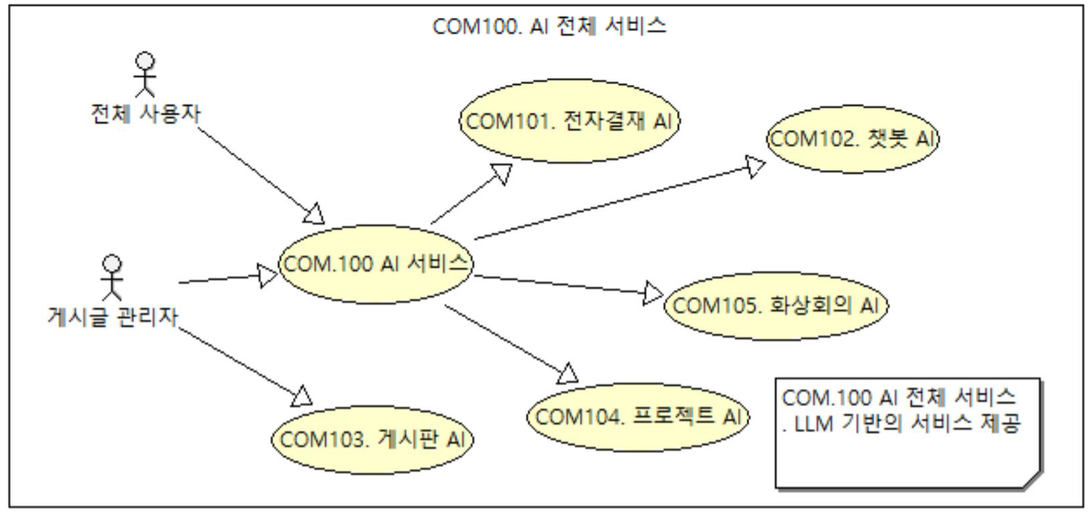
PROJECT 02 · 분석 · 설계

유스케이스 & 시퀀스 다이어그램

요구사항을 행위자·기능으로 정리하고, 핵심 흐름을 시퀀스로 설계했습니다.

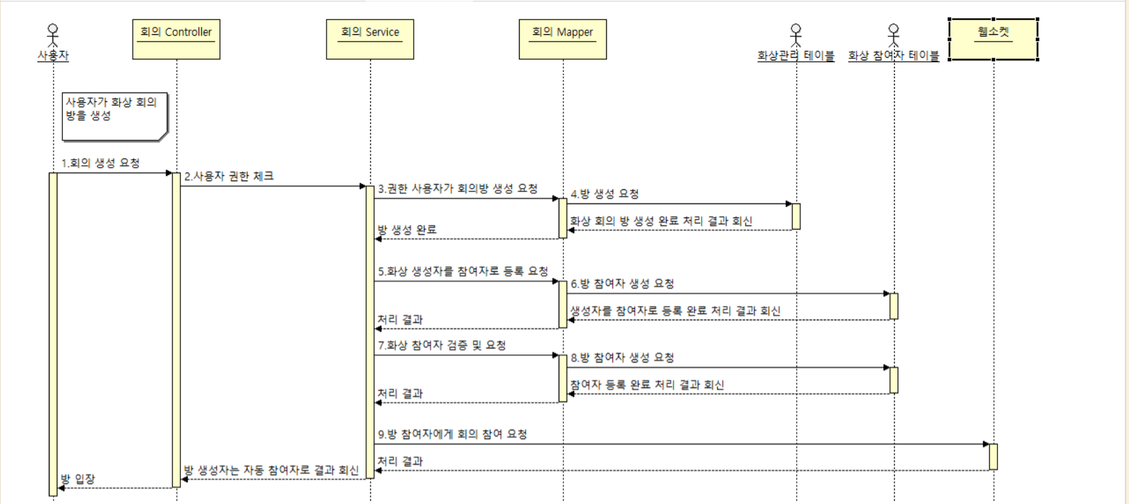
USE CASE 유스케이스 다이어그램

usecase.png



SEQUENCE 시퀀스 다이어그램

sequence.png



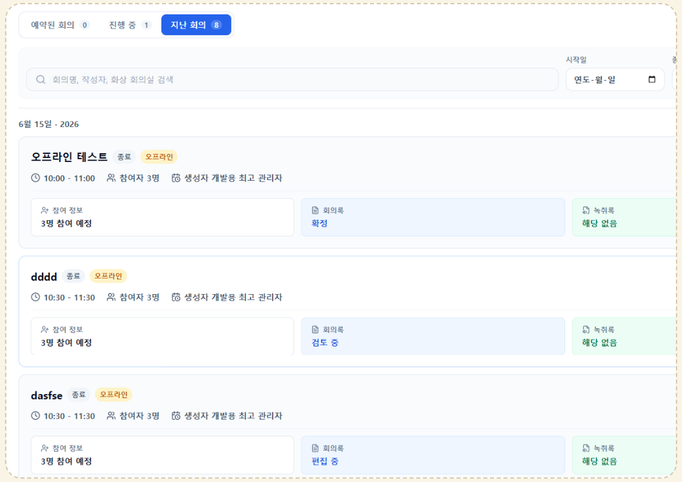
PROJECT 02 · 상세 설계

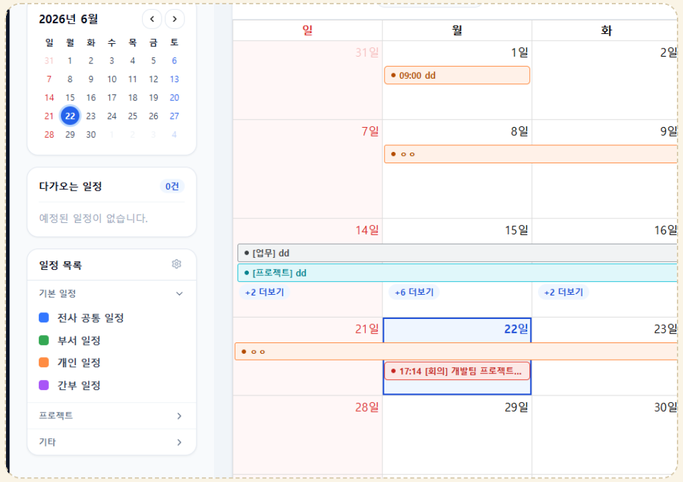
API 명세 & 화면설계서

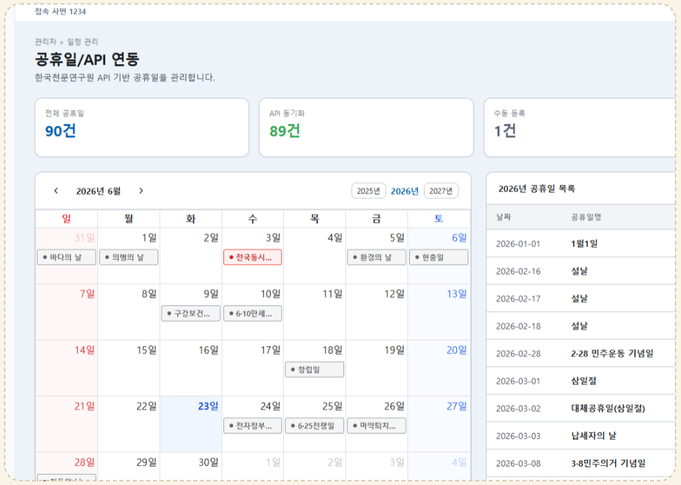
API 명세 (양식)

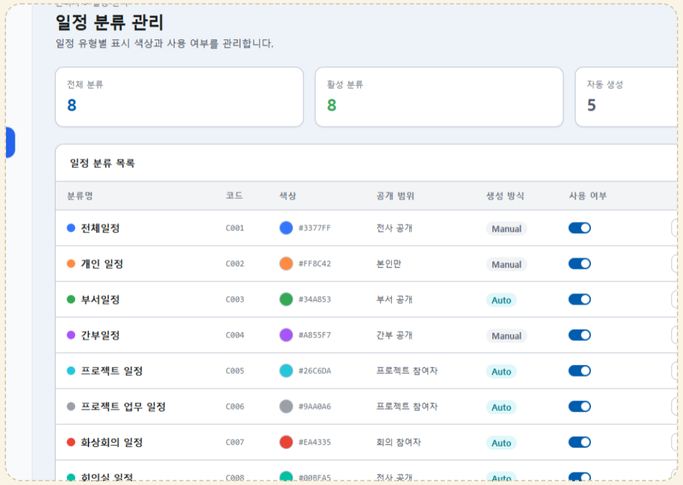
Method	Endpoint	설명
POST	/api/meetings	회의 생성
GET	/api/meetings/{mtngId}/minutes	회의록 조회 (AI 자동 생성)
PUT	/api/meetings/{mtngId}/minutes	AI 생성 회의록 수정
POST	/api/dashboard/widgets/{type}	위젯별 독립 데이터 조회
POST	/api/search?keyword=	6개 도메인 통합검색
PUT	/api/notifications/subscribe	SSE 실시간 알림 구독
DEL	/api/reservations/{id}	회의실 예약 취소

화면설계서 (와이어프레임)



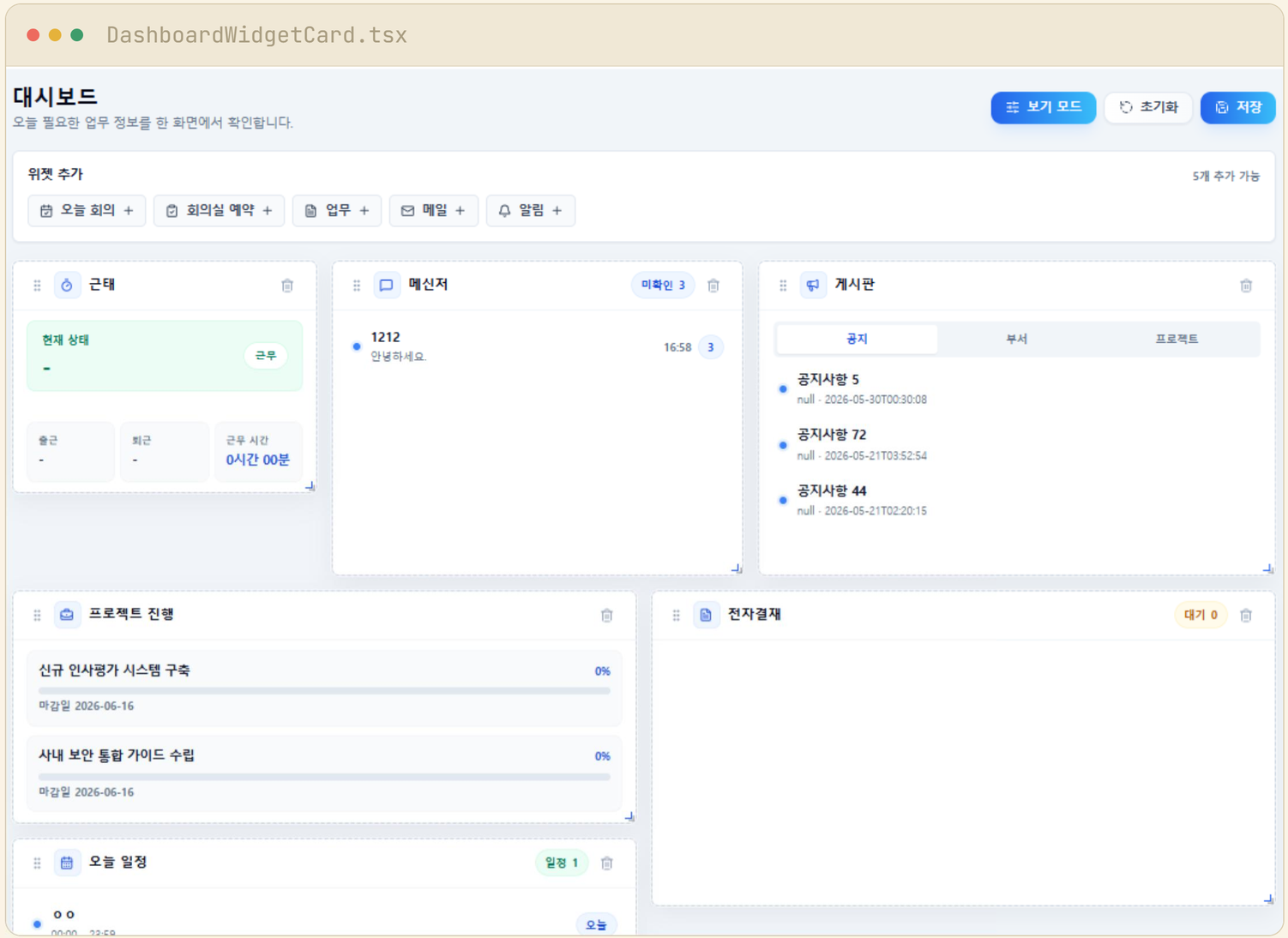






PROJECT 02 · 주요 기능

핵심 기능 상세



01 **직원별 JSON CLOB 레이아웃 저장**

위젯 배치 정보를 JSON CLOB으로 저장해 직원마다 독립적인 레이아웃을 유지합니다. 위젯 추가 시 테이블 스키마 변경 없이 JSON 구조만 확장하면 되어 유연한 확장이 가능합니다. 직원이 재배치한 위젯은 다음 접속 시 그대로 복원됩니다.

02 **위젯별 독립 API 부분 렌더링 최적화**

12종 위젯을 각각 독립 엔드포인트로 분리했습니다. 하나의 위젯에 오류가 발생해도 다른 위젯에 영향을 주지 않으며, 프론트에서 병렬 호출로 전체 로딩 시간을 단축합니다. 필요한 위젯 데이터만 선택적으로 갱신할 수 있습니다.

03 **역할별 기본 레이아웃 코드 관리**

최초 접속 사용자의 기본 레이아웃을 DB가 아닌 Java 코드에서 역할별로 정의했습니다. DB 초기 데이터 없이도 역할에 맞는 위젯 구성이 자동 생성되며, 기본 레이아웃 변경 시 SQL 없이 코드 수정만으로 반영됩니다.

PROJECT 02 · AI PIPELINE

AI 회의록 자동화 파이프라인

음성 인식부터 회의록 생성까지, **외부 AI API 2종**(STT·LLM)을 직접 연동했습니다.

01
음성 캡처
LiveKit
화상회의 오디오 스트림 수집

→

02 EXTERNAL API
음성 → 텍스트
Whisper STT
5초 단위 실시간 자막 변환

→

03
대화 로그 누적
Conversation Log
발언자·시간 단위 타임라인 저장

→

04 EXTERNAL API
회의록 생성
LLM
로그 기반 구조화 회의록 HTML

→

05
비동기 저장·알림
@Async
백그라운드 후처리 후 알림 전송

ChatGptController.java · LLM 회의록 스트리밍 제어

```java
Flux<String> origin = chatClient.prompt()
        .user(aiMessage)
        .stream()
        .content()
        .delayElements(Duration.ofMillis(20));

return origin
        .takeUntil(chunk -> apiSupportManager.shouldStop(requestId))
        .onErrorResume(ex -> {
            log.warn("Stream aborted: {}", ex.getMessage());
            return Flux.empty(); })
        .doFinally(signalType -> apiSupportManager.clear(requestId));
        //LLM 응답을 SSE로 스트리밍하고, 중단 요청 시 안전하게 종료
```

근거 기반 회의록 생성 구조

AI 회의록 생성 시 **실제 회의에서 발화되지 않은 내용을 임의로 보완하는 환각 문제**가 발생할 수 있다고 판단했습니다. 이를 방지하기 위해 프롬프트를 `[ROLE]·[RULES]·[REFERENCE]·[OUTPUT FORMAT]` 구조로 분리하고, **회의 대화 로그만 근거로 사용**하도록 제한했습니다.

특히 `REFERENCE` 정보는 참고용으로만 사용하고, 회의록 본문에는 **발화 로그에 존재하는 내용만 반영**하도록 설계했습니다. 또한 결과 형식을 `HTML` 구조와 `{SIGN:N}` 서명 토큰으로 고정하여 전자결재 문서로 바로 전달할 수 있도록 했습니다.

AI 초안 → 전자결재 연동 → 회의 후속 업무 자동화

회의 종료 후 생성된 AI 회의록 초안은 **담당자 검토 단계**를 거쳐 전자결재 기안 문서로 전환되도록 설계했습니다. 초안 상태는 `01. AI초안 → 02. 편집중 → 03. 결재요청` 흐름으로 관리하여, **AI가 생성한 결과를 그대로 확정하지 않고 사람이 검토한 뒤** 결재 프로세스로 넘기도록 했습니다.

이를 통해 회의 녹취, `STT` 변환, `LLM` 회의록 생성, 담당자 검토, 전자결재 기안까지 이어지는 후속 업무를 **하나의 자동화 파이프라인으로 연결**했습니다.

PROJECT 02 · 핵심 코드

코드 하이라이트

StompChannelInterceptor.java

```java
@Override
public Message<?> preSend(Message<?> message, MessageChannel channel) {
    StompHeaderAccessor accessor =
            MessageHeaderAccessor.getAccessor(message, StompHeaderAccessor.class);
    // 최초 WebSocket 연결(CONNECT) 요청에서만 JWT 인증 수행
    if (accessor != null && StompCommand.CONNECT.equals(accessor.getCommand())) {
        String authHeader = accessor.getFirstNativeHeader("Authorization");
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            throw new CustomException(ErrorCode.WS_UNAUTHORIZED);
        }
        String token = authHeader.substring(7);
        jwtTokenProvider.validateToken(token);
        // 검증된 토큰에서 사원 ID를 추출해 WebSocket 사용자 정보로 등록
        Long empId = jwtTokenProvider.getEmpId(token);
        accessor.setUser(() -> String.valueOf(empId));
    }
    return message;
}
```

무엇을 하는 코드인가

WebSocket STOMP 연결(CONNECT) 시점에 Authorization 헤더에서 JWT를 추출해 검증하고, 사원 ID를 `Principal`로 등록합니다. 이후 SEND·SUBSCRIBE 메시지에서도 별도 인증 없이 사원 ID를 그대로 사용할 수 있습니다.

핵심 포인트

HTTP 기반 Spring Security 필터는 WebSocket 환경에서 동작하지 않아 `ChannelInterceptor`를 직접 구현했습니다. `StompHeaderAccessor.wrap()` 대신 `MessageHeaderAccessor.getAccessor()`를 사용한 이유는, wrap은 새 객체를 만들어 헤더 수정이 원본 메시지에 반영되지 않지만 getAccessor는 기존 객체를 그대로 반환해 `setUser()` 결과가 실제 메시지에 적용되기 때문입니다.

POINT — HTTP 보안 필터가 닿지 않는 WebSocket 영역에 별도 인증 흐름을 설계했습니다.

PROJECT 02 · 핵심 코드

코드 하이라이트

MessengerEventListener.java

```java
@Component
@RequiredArgsConstructor
public void MessengerEventListener {
    private final MsngrService msngrService;
    private final ProjectService projectService;

    @Async
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void MessengerEventListener(ProjectCreatedEvent event) {
        try{
            //메신저 방 자동 생성
            Long chtrmId = msngrService.createProjectChtrm(...);
            //생성된 방 ID를 프로젝트 테이블에 업데이트
            projectService.updateProjectChtrmId(event.getProjId, chtrmId);
        } catch (Exception e) {
            log.error("메신저 채팅방 자동 생성 실패", e);    //예외(에러) 로그
    }
}
```

무엇을 하는 코드인가

프로젝트 생성 시 DB 커밋이 완료된 직후, 해당 프로젝트의 메신저 채팅방을 자동으로 생성하고 방 ID를 프로젝트 테이블에 업데이트합니다. 사용자는 별도 설정 없이 프로젝트 생성과 동시에 전용 채팅방에 입장할 수 있습니다.

핵심 포인트

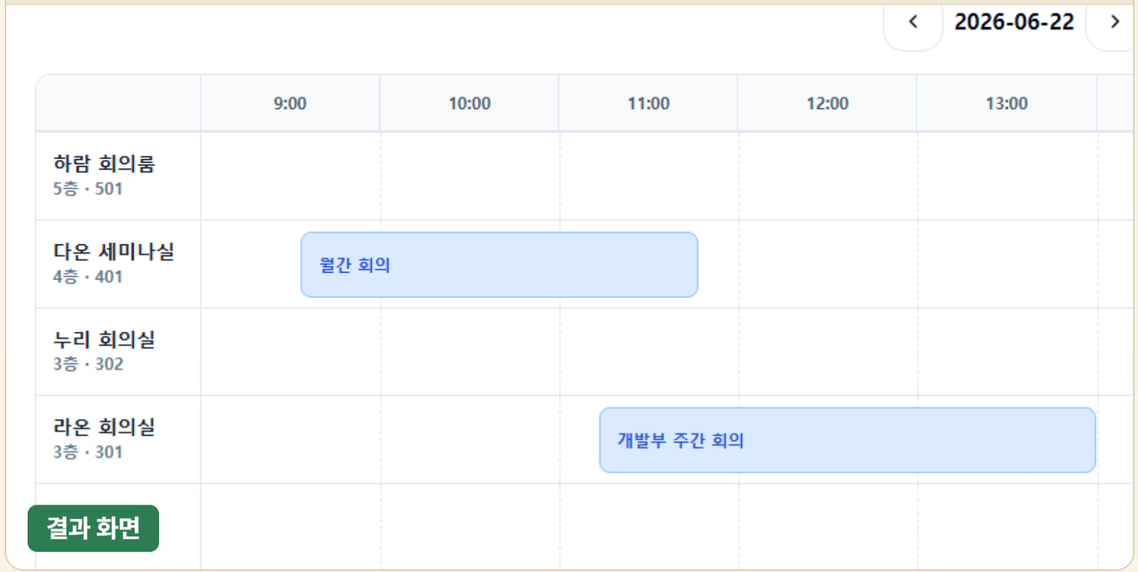
`@TransactionalEventListener(AFTER_COMMIT)`으로 DB 커밋 완료 후에만 채팅방이 생성되어 트랜잭션 롤백 시 프로젝트 없이 채팅방만 따로 생성되어 남는 문제를 방지했습니다. 프로젝트 서비스는 이벤트를 발행하는 것만 담당하고, 메신저 채팅방 생성 로직은 리스너에 완전히 위임해 도메인 간 결합력을 낮췄습니다.

POINT — DB 커밋 이후 채팅방을 생성하도록 분리해,
프로젝트 생성과 메신저 연동의 데이터 정합성을 확보했습니다.

PROJECT 02 · 화면 코드 리뷰

화면 ↔ 코드 매핑

ReservationPage.tsx



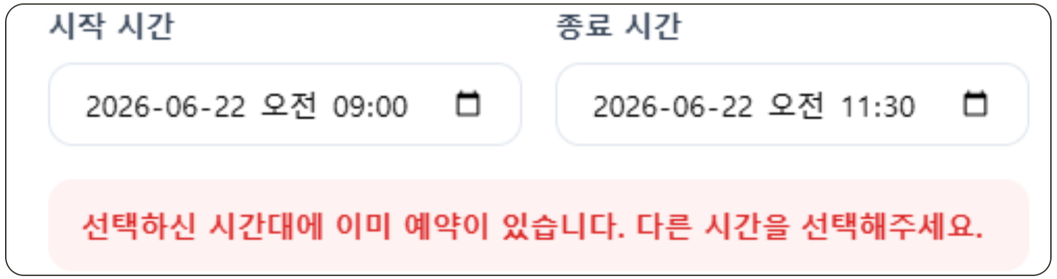

```xml
<select id="countOverlappingRsrv" resultType="int">
    SELECT COUNT(*)
    FROM TB_CONF_RM_RSRV
    WHERE CONF_RM_ID = #{confRmId}
      AND DEL_YN = 'N'
      AND RSRV_ID != #{rsrvId}
      AND BEGIN_DT  <![CDATA[ < ]]> #{endDt}
      AND END_DT    <![CDATA[ > ]]> #{beginDt}
</select>
```

처리 흐름

Client → Controller → Service → **Mapper** → DB

상세 설명

회의실 예약 등록 시, 같은 회의실의 동일 시간대 중복 예약을 **DB 쿼리 레벨에서 차단**합니다. 사용자가 회의실과 시간을 선택하면 Controller가 요청을 받아 Service로 전달하고, Service는 Mapper를 통해 **countOverlappingRsrv** 쿼리를 실행해 같은 회의실(CONF_RM_ID)·같은 시간대에 정상 예약(DEL_YN = 'N')이 존재하는지 확인합니다. 구간 겹침 판정 조건은 **BEGIN_DT < 요청종료 AND END_DT > 요청시작**으로, **등호를 제외**해 딱 붙는 예약(9시~10시 / 10시~11시)은 허용합니다. 수정 요청의 경우 본인 예약(RSRV_ID != #{rsrvId})을 제외해 자기 자신과의 충돌을 방지합니다. 중복이 감지되면 **INSERT 없이 즉시 예외**를 반환하고, 프론트에 에러 메시지를 표시합니다.



PROJECT 02 · 화면 흐름

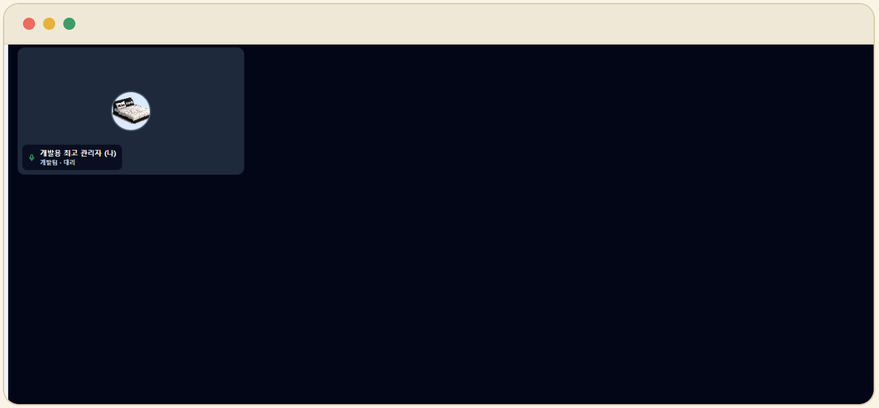
기능 시나리오

온라인 회의 생성부터 AI 회의록 자동 생성, 전자결재까지의 핵심 사용자 흐름입니다.



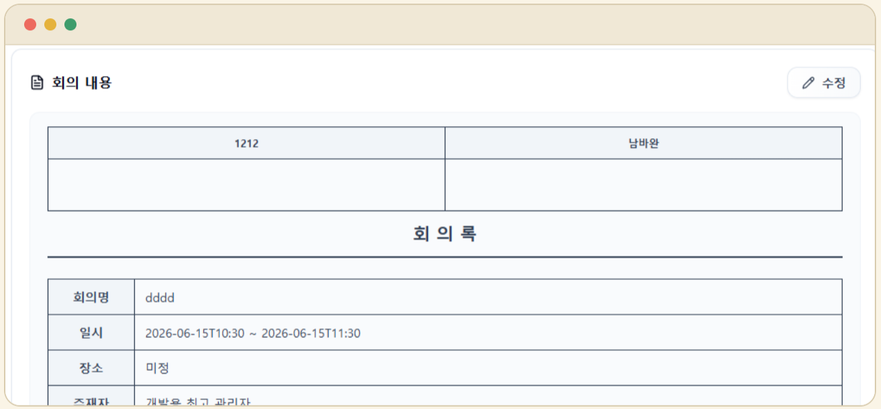
STEP 1 화상회의 입장

회의 생성 시 진행 방식(온라인/오프라인/혼합)과
참여자, 회의실을 설정하고 화상회의에 입장합니다.



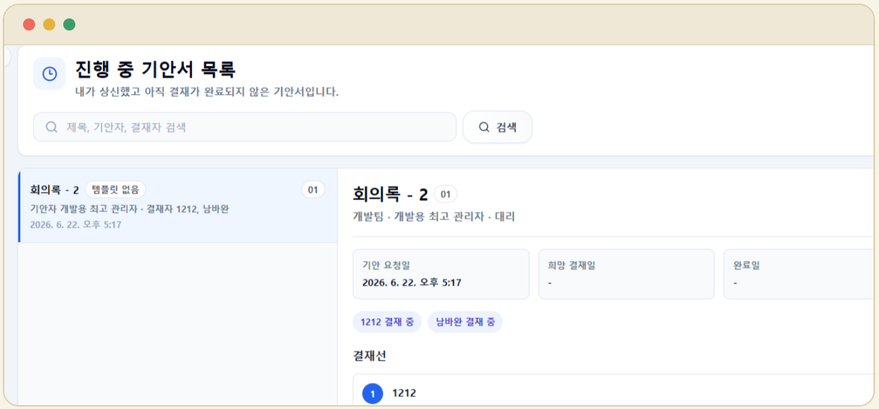
STEP 2 화상회의 진행

발언이 5초 단위로 **외부** `Whisper STT API`로 텍스트 변환되어
자막으로 표시되고 대화 로그로 누적됩니다.



STEP 3 회의록 자동 생성

회의 종료 시 누적된 대화 로그를 기반으로 `LLM`이
회의록 HTML을 **비동기로** 자동 생성합니다.



STEP 4 전자결재 상신

참여자가 회의록 검토 후 결재 요청하면 전자결재에
기안이 생성되고, 승인 완료 시 회의록이 확정됩니다.

PROJECT 02 · TROUBLESHOOTING

트러블슈팅

CASE 01

문제

화상회의와 회의실 예약을 별도 기능으로 설계해 회의록/참석자/결재가 화상회의에 종속됨

발견 계기

기능 구현을 마친 후 현직 개발자 지인에게 실제 업무 프로세스를 들었습니다. 오프라인 회의에서도 회의록을 작성해 결재받아야 하고, 화상회의도 회의실에서 진행하는 경우가 많다는 사실을 그제야 인식했습니다. **구현 기술(LiveKit)을 기준으로 설계하다 보니 "회의"라는 상위 도메인 개념 자체가 빠진 것이 근본 원인이었습니다.**

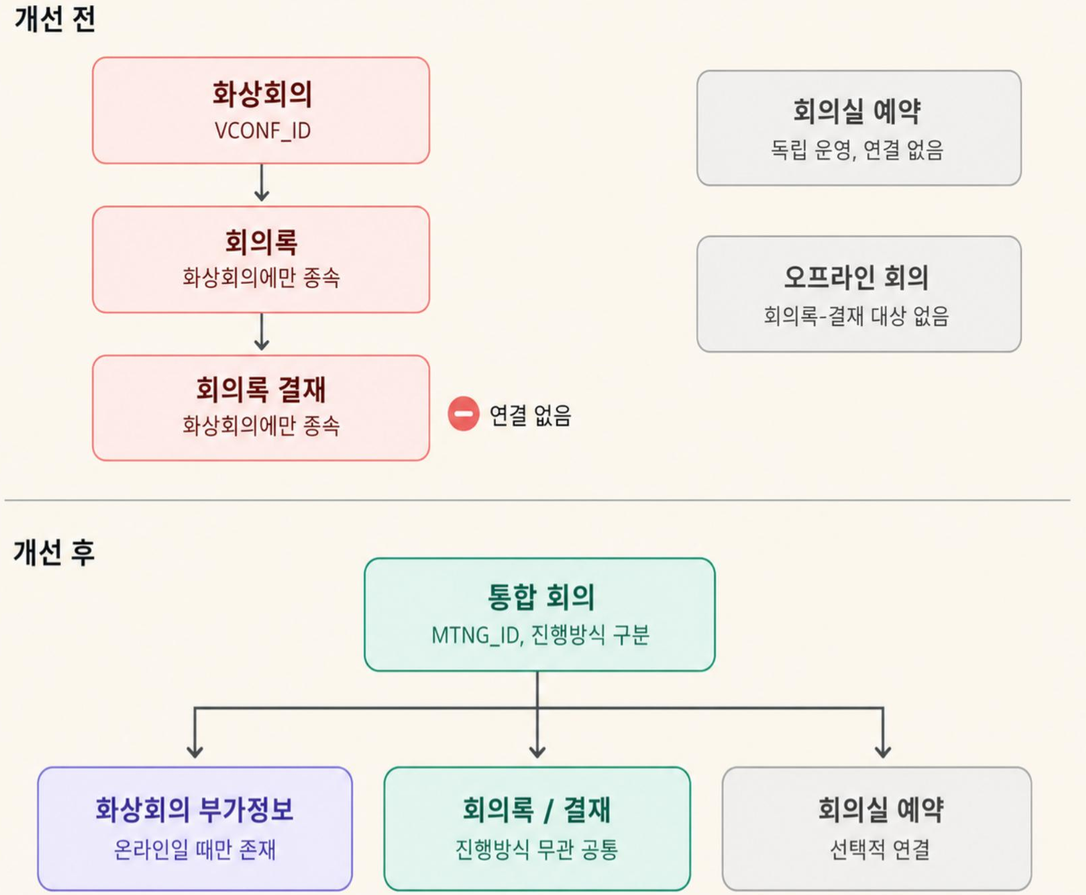
해결

`TB_MTNG`를 상위 도메인으로 신설해 온라인/오프라인 회의를 통합했습니다.

- 회의 통합: `MTNG_TYPE_CD`로 온라인/오프라인/혼합 구분
- 회의록 재구성: `TB_VIDEO_MOM` → `TB_MTNG_MOM`, `MTNG_ID` 기준 관리
- 참석자 분리: 참석자 명단과 입퇴장 로그 분리
- AI 회의록 처리: `@Async` + `AFTER_COMMIT`으로 별도 처리해 응답 지연 방지

배운 점

실제 업무 프로세스 없이 구현 기술 기준으로 설계하면 기술적으로는 작동하지만 현실과 맞지 않는 구조가 나오게 됩니다. "이 기능이 다른 진행 방식에서도 재사용될 수 있는가?"를 설계 단계에서 먼저 질문하는 습관을 가지게 되었습니다.



PROJECT 02 · TROUBLESHOOTING

트러블슈팅

CASE 02

문제

통합 검색창 타이핑 시 실시간 자모음 합성으로 인해 무분별한 중복 API 요청이 발생함. 단어 하나 입력당 10회 이상의 대형 쿼리가 유입되어 서버 및 DB 과부하를 유발함.

발견 계기

기능 구현 후 프론트엔드 연동 테스트 과정에서 입력 속도에 비례해 화면이 미세하게 버벅거리는 현상을 인지하였습니다. 백엔드 콘솔에 동일한 통합 검색 SQL 로그가 순간적으로 폭발하듯 찍히는 것을 보고 성능 병목 위험을 확인하였습니다.

해결

입력 이벤트 지연을 위한 **250ms 디바운싱(Debouncing)** 기법을 컴포넌트에 도입.

- **타이머 제어**: `useEffect`와 `setTimeout`을 활용해 타자 입력 중인 중간 요청을 물리적으로 차단
- **부하 절감**: 사용자 입력이 완전히 끝난 시점에만 최종적으로 단 1회 요청하도록 제어

배운 점

크롬 개발자 도구(Network 탭) 실측 결과 **API 호출 횟수가 80% 이상 절감**됨을 확인하였습니다. 대량의 데이터를 다루는 `LIKE` 검색 특성상 연쇄적인 Full Table Scan 오버헤드를 사전에 차단하는 것이 중요함을 깨달았습니다. 단순 기능 구현을 넘어 서버와 DB 리소스를 고려하는 설계 시야를 갖추게 되었습니다.



THANK YOU

흐름을 만들고
끝까지 개선합니다.

사용자 경험과 시스템 안정성을 함께 고민하는
풀스택 개발자로 성장하겠습니다.

EMAIL
sksqlrlek@gmail.com

PHONE
+82 010-4614-5056

GITHUB
github.com/sksqlrlek